

Un agente investigador sobre informes 10-K

MIAX · Instituto BME · Bloque V — LLMs aplicados a Finanzas Pietro, Alonso y Raúl

Este documento se genera con `agente-10k informe --pdf`. Todas las tablas salen de `resultados/`, que a su vez sale de ejecutar el repositorio. No hay ninguna cifra escrita a mano: si una tabla falta, aquí aparece qué orden la genera.

1. Qué hace el sistema

Un agente ReAct responde preguntas sobre los 10-K de seis tecnológicas (NVDA, MSFT, AAPL, GOOGL, META, AMZN) en FY2024 y FY2025 — 48 secciones, unos 650.000 tokens — y cita de dónde sale cada dato. Tiene cuatro herramientas con coste y precisión muy distintos:

herramienta	qué da	coste
<code>list_available</code>	qué emisores y ejercicios hay	gratis
<code>get_xbrl_fact</code>	el hecho XBRL exacto	gratis y exacto
<code>search_filings</code>	los fragmentos más parecidos a una consulta	barato, difuso
<code>read_section</code>	una sección entera, hasta cuarenta páginas	caro

La tarea central no es contestar: es **elegir bien**. Una cifra de los estados financieros se pide a XBRL, donde es exacta y gratuita; un matiz de los factores de riesgo hay que buscarlo en el texto. Un agente que lee la sección 8 entera para dar un beneficio por acción ha acertado por el camino equivocado, y en nuestra tabla eso cuenta como fallo.

La respuesta sale siempre como `RespuestaFinanciera`: prosa, cifra, unidad, emisor, ejercicio, fuente, cita literal y `chunk_id`. Añadimos dos campos — `concept_xbrl` y `motivo_sin_dato` — que el contrato permite porque llevan valor por defecto: el primero deja que el guardarraíl contraste contra el hecho exacto en lugar de adivinar qué concepto quiso decir el modelo, y el segundo distingue «no está en el corpus» de «no lo he sabido encontrar», que no es lo mismo.

2. El golden set

Veinte preguntas en `golden/golden_set.jsonl`, repartidas entre extractivas, numéricas y comparativas, con dos huecos deliberados: preguntas cuyo dato **no está** en el corpus y cuya respuesta correcta es decirlo.

La verdad de las extractivas se ancla a una **frase literal** del informe, nunca a un `chunk_id`. El identificador cambia en cuanto se re-trocea el corpus, y quien mejore el troceado no puede salir penalizado por haberlo mejorado. El `recall@k` se mide comprobando que esa frase aparece en alguno de los `k` fragmentos recuperados **y que el fragmento es del emisor y el ejercicio que la pregunta pide**: sin esa segunda comprobación, una frase que se repite en dos informes contaría como acierto en el equivocado.

`golden/validador.py` comprueba el esquema, las familias, los huecos y las escalas antes de que ninguna pregunta entre en una medición.

3. Cómo se evalúa

Tres evaluadores independientes miran cada respuesta:

- **cita** — el texto citado tiene que existir literalmente en el fragmento que dice; se compara con espaciado y comillas normalizados, no carácter a carácter;
- **cifra** — se acepta con una tolerancia única, 0,5 % o 1 USD, la que sea mayor (ADR-008), y se comprueba la unidad y la escala: confundir millones con miles es el fallo más frecuente y el más silencioso;
- **trayectoria** — qué herramientas usó y en qué orden.

El **acierto** exige las dos cosas: respuesta correcta **y** camino correcto. La tabla lleva además la columna de *acierto por el camino equivocado*, que es la que enseña cuánto de la nota es suerte.

Las preguntas hueco tienen su propia columna y su propio fallo: **alucinación sobre hueco**, dar una cifra donde la respuesta correcta era que el dato no está. Y su simétrico, la **abstención indebida**: decir «no hay dato» cuando sí lo había. Un sistema que no contesta nunca no alucina y no vale para nada; hacen falta las dos columnas para que ninguna de las dos trampas pase desapercibida.

Veinte preguntas no son una muestra grande

Con veinte preguntas, subir de 12 a 14 aciertos es mover dos preguntas. Por eso cada proporción va con su **intervalo de Wilson** —no la normal, que con n pequeño se sale de $[0, 1]$ y da anchura cero cuando se acierta todo— y cada comparación entre dos sistemas se contrasta con **McNemar exacto**: pareado, porque los dos sistemas responden las mismas preguntas, y exacto, porque la aproximación de chi-cuadrado pide unos veinticinco pares discordantes y aquí hay cuatro o cinco.

4. El retrieval, paso a paso

Cada fila es una configuración; el runner es el mismo. Añadir una fila a esta tabla es añadir una tupla en `retrieval/fabrica.py`, nunca escribir código.

configuración	recall@1	recall@3	recall@5	recall@10	MRR	latencia media (s)	coste medio (\$)
denso (base)	0.00	0.00	0.25	0.25	0.06	751.8 ms	0.00000
+ filtro metadatos	0.25	0.50	0.67	0.83	0.44	252.8 ms	0.00000
+ BM25 (híbrido RRF)	0.25	0.50	0.58	0.67	0.40	256.4 ms	0.00000
+ reescritura de consulta	0.67	0.67	0.75	0.92	0.71	4904.4 ms	0.00192
todo	0.58	0.83	0.92	1.00	0.74	4920.3 ms	0.00192
+ reordenación (cross-encoder)	0.17	0.25	0.58	0.75	0.32	711.3 ms	0.00000
+ reescritura + reordenación	0.83	0.83	0.83	0.92	0.85	5198.3 ms	0.00192

configuración	recall@1	recall@3	recall@5	recall@10	MRR	latencia media (s)	coste medio (\$)
todo + reordenación	0.83	0.83	0.83	0.92	0.85	5212.9 ms	0.00192
+ reescritura + reordenación (RRF)	0.67	0.75	0.92	0.92	0.74	5196.7 ms	0.00192
todo + reordenación (RRF)	0.67	0.92	0.92	0.92	0.78	5167.6 ms	0.00192

Y el contraste de cada fila contra la de partida:

Contraste contra «denso (base)» sobre recall@5, 12 preguntas con ancla. McNemar exacto, pareado.

configuración	recall@5	arregla	rompe	p
+ filtro metadatos	0.67	5	0	0.062 (n. s.)
+ BM25 (híbrido RRF)	0.58	4	0	0.125 (n. s.)
+ reescritura de consulta	0.75	6	0	0.031
todo	0.92	8	0	0.008
+ reordenación (cross-encoder)	0.58	4	0	0.125 (n. s.)
+ reescritura + reordenación	0.83	7	0	0.016
todo + reordenación	0.83	7	0	0.016
+ reescritura + reordenación (RRF)	0.92	8	0	0.008
todo + reordenación (RRF)	0.92	8	0	0.008

Lectura:

- **El filtro de metadatos es lo primero y casi lo más barato.** Filtrar *antes* de buscar, en vez de filtrar el orden que devuelve el índice, sube el recall sin coste ni latencia. Aun así no aguanta el contraste por sí solo: con doce preguntas con ancla, cinco aciertos nuevos y ningún fallo nuevo se quedan justo en el borde.
- **La reescritura de la consulta es la mejora grande, y cuesta dinero.** La pregunta llega en español y en lenguaje de persona; el corpus está en inglés y en lenguaje de abogado. Traducirla con el modelo antes de buscar es lo que parte la tabla en dos, y es la primera fila cuyo p-valor baja del 5 %. Añade una llamada al modelo y unos cinco segundos por consulta: está en la columna de coste y en la de latencia, no en una nota al pie.
- **El cross-encoder solo vale después de traducir.** Reordenar los candidatos con `ms-marco-MiniLM-L6-v2` sobre la consulta en español *empeora* —el modelo está entrenado en inglés—; sobre la consulta ya traducida sube el recall@1 y el MRR de forma clara. Es el mejor ejemplo de la práctica de que el orden de las mejoras importa tanto como las mejoras.
- **Reordenar puro sube arriba y puede hundir abajo.** El cross-encoder no recupera, solo reordena: cuando se equivoca, tira fuera del top-5 un pasaje que el recuperador ya tenía. Fusionar su orden con el del recuperador por RRF —el mismo RRF del híbrido— conserva el recall profundo y sigue ganando arriba. Para un agente que lee cinco fragmentos, la fila de RRF es la que hay que llevar.
- **BM25 no aportó.** El híbrido denso + léxico con RRF no mejora al denso con filtro en nuestro golden set. Es un arreglo razonable que no movió la métrica, y eso también es un resultado.

La tabla es **reproducible**: las reescrituras se cachean en disco con el modelo en la clave, porque el modelo no devuelve la misma reescritura ni con temperatura 0, y sin esa caché dos ejecuciones seguidas daban recalls distintos. Los aciertos de caché siguen sumando su coste y su latencia originales: si no, la segunda ejecución diría que reescribir es gratis.

5. Baseline contra sistema final

El baseline es el agente del día 10 — `miax_s2.baseline()` — con **el mismo modelo** que el sistema final: la tabla compara dos sistemas, no dos modelos. Se ejecutó y se **congeló** antes de tocar nada (`resultados/baseline/SELLO.json`, huellas SHA-256 de cada fichero); regenerarlo exige borrar el sello a mano y dejar constancia en `docs/decisiones.md`.

PENDIENTE: falta docs\informe\tabla_principal.md. Se genera ejecutando el repositorio.

PENDIENTE: falta docs\informe\significancia.md. Se genera ejecutando el repositorio.

Por familia y por pregunta

PENDIENTE: falta docs\informe\resultados_baseline.md. Se genera ejecutando el repositorio.

6. Las diez preguntas ciegas

PENDIENTE: falta docs\informe\delta_ciegas.md. Se genera ejecutando el repositorio.

Si el delta baja, la parte de la mejora que era memoria del conjunto con el que iteramos se ve aquí. Es un hallazgo, no un suspenso: lo que no se puede hacer es no medirlo.

7. Reproducir esto

```
pip install -e ".[dev,informe]"
# descomprimir corpus_miax_2026.zip e indice_faiss.zip en data/corpus/
agente-10k diagnostico      # qué hay y qué falta
agente-10k validar-golden   golden/golden_set.jsonl
agente-10k ablacion         # tabla de retrieval + contraste
agente-10k baseline        # ejecuta y CONGELA el baseline
agente-10k evaluar golden/golden_set.jsonl --etiqueta final
agente-10k informe --pdf    # este documento
```

Ninguna clave de API está en el repositorio: se leen del entorno o de un `.env` que no se versiona, y `.env.example` dice cuáles hacen falta.